AstroBug: Automatic Game Bug Detection Using Deep Learning

Elham Azizi and Loutfouz Zaman

***Abstract*—Traditional methods of video game bug detection, such as manual testing, have been effective, but they can also be time-consuming and costly. While automated bug detection techniques hold great promise for improving testing, they still face several challenges that need to be addressed to be effective in practice. In this work, we introduce a new framework to detect perceptual bugs using a long short-term memory network, which detects bugs in games as anomalies. The detected buggy frames are then clustered to determine the category of the occurred bug. The framework was evaluated on two first person shooter games. We further enhanced the framework by implementing a reinforcement learning agent to autonomously gather datasets, effectively addressing the need for human players to collect data and manually browse through games. The enhancement was performed on a role-playing game. The outcomes obtained validate the effectiveness of the framework.**

***Index Terms*—Anomaly detection, automatic bug detection (ABD), game testing, long short-term memory (LSTM) network, reinforcement learning (RL).**

I. Introduction

VIDEO games are designed to be an immersive experience for the user. However, the level of immersion a player experiences is drastically diminished if bugs are discovered during play [1]. Bugs in games can have various effects, ranging from minor annoyances to significant negative impacts on player experience, game sales, and the reputation of developers that prevent players from advancing or completing certain objectives. Before the games are launched, the industry directs resources and puts effort to minimize the impact of bugs on games [2]. However, even large developers working on AAA games end up in a situation where significant bugs are slipping through. While developers strive to release bug-free games, the complexity and scale of modern games can make it challenging to catch and eliminate all the bugs before release. As a result, playtesting these large and complex games using human testers is becoming unreasonable [2]. Moreover, traditional methods of bug detection, such as manual testing, have been effective, but they can also be time-consuming and costly [3]. Furthermore, the sheer variety of bugs that can arise is vast, making it impractical to test a game comprehensively at every level, particularly for complex and expansive video games. The emergence of bugs can vary based on the level at which a bug occurs, these are: low-level, engine, application, perceptual, and behavioral [4]. Fig. 1 provides a visual representation of the bug levels, with increasing difficulty in bug identification as one moves up the hierarchy.

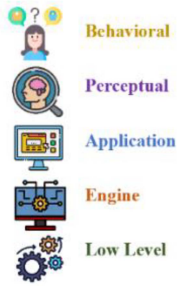

Fig. 1. Hierarchy of bug identification based on automation and identifying difficulty.

Perceptual and behavioral bugs cannot be detected by defining logical rules to infer issues trivially. Perceptual bugs in games refer to issues that affect the appearance or display of game elements whereas behavioral bugs affect the game logic or mechanics. With the rise of artificial intelligence (AI), game developers now have a new tool to detect bugs in their games [5], [6], [7]. AI can help automate and enhance the bug detection process, making it more efficient and effective.

To effectively address the problem of game bug detection and build upon the limitations of the previous works, we identified the key requirements that the solution needs to fulfill to be effective and efficient. These requirements include the following:

1) automatic and accurate detection of perceptual and behavioral bugs;
2) the use of an unsupervised learning strategy to autonomously identify data patterns and structures, eliminating the need for prelabeled datasets and thereby avoiding bug category omission and reducing data handling efforts;
3) the design of the solution as well as the data structure should be generalizable;
4) the solution should be able to handle buggy frames where the bugs' presence would not be detectable without considering the preceding frames;
5) ideally, the solution should categorize detected bugs, enabling developers to prioritize fixes, formulate targeted solutions, enhance team communication, and expediently address issues, especially since certain bug categories in games can recur across various scenes and levels.

Manuscript received 16 October 2023; revised 26 March 2024; accepted 8 May 2024. Date of publication 17 May 2024; date of current version 17 December 2024. This work was supported by NSERC Discovery. *(Corresponding author: Elham Azizi.)*

The authors are with the Faculty of Business and Information Technology, Ontario Tech University, Oshawa, ON ON L1G 0C5, Canada (e-mail: elham.azizi@ontariotechu.net; loutfouz.zaman@ontariotechu.ca).

Digital Object Identifier 10.1109/TG.2024.3402626

To build a system with above specifications, we present a novel AI-based framework, named AstroBug, for bug identification.[1] In this work, we use long short-term memory (LSTM) [8]—a deep neural network (DNN) architecture for behavioral and perceptual bug detection. We frame the bug detection problem as anomaly detection [9], which allows integrating the framework to different video games easily. In the context of video analysis, anomaly detection involves identifying unusual or abnormal events, behaviors, or patterns that deviate from the expected norm. In the first round of this study, our approach is tested on two 3-D first person shooter (FPS) games. After finding an anomaly in video frames, we specify the type of bug presented in the frame using the density-based spatial clustering of applications with noise (DBSCAN) algorithm [10]. This section of the algorithm is also referred to as bug identification, which involves identifying the type of bug occurred. In the second round of the study, we trained a reinforcement learning (RL) [11] agent to perform the task of browsing and collecting datasets within a third game, which is a role-playing game (RPG). This allows the framework to automate the dataset generation phase and removes the dependence on human game players. This also expands upon the game play and test coverage. The key contributions made by our work are as follows:

1) a new automatic framework for full perceptual and behavioral bug detection;
2) a generalizable approach to addressing bug detection problems;
3) handling buggy frames that may not be recognizable without considering the preceding frames;
4) an introduction of a bug identification method using a semisupervised algorithm.

This work substantially enhances our previously published short paper [12] by offering an exhaustive literature review, providing detailed insights into the framework, and incorporating a second study with an RL agent.

II. Related Works

A. Traditional Game Testing

Traditional game testing methods encompass manual assessments and observation sessions conducted by human testers to identify bugs. Manual testing is dependent on human testers executing test cases and actively looking for potential bugs.

Most of the manual approaches for game testing consider usability evaluations on games. Choi [13] explored whether usability evaluation for games face validity in game development. Choi conducted a case study and identified that the usability expert evaluation provides novel and useful feedback for game testing. In a study conducted by Korhonen and Koivisto [14], a novel model for playability heuristics was proposed and it was specifically tailored for mobile games. The model includes three main modules, which are game usability, mobility, and game play. They validated the effectiveness of their gameplay heuristics model in enhancing the game evaluation process and identifying playability issues. A new methodology by Moreno-Ger et al. [15] was designed to facilitate game usability testing. The methodology suggests a list of improvements based on a thorough analysis of many recorded gameplays. Their approach primarily benefits playtesting of serious games by providing a more systematic and targeted approach to generating improvements. Diah et al. [16] tested their new introduced usability testing method on a game called *Jelajah* with children. Since usability is an important concept in designing games for children, they conducted a study to observe the effectiveness of the game within their introduced framework. The results obtained from the collected data showed that their evaluation method can be adopted effectively.

B. Automated Game Testing

Automated bug detection (ABD) research primarily revolves around the creation of software agents with the ability to 1) explore and engage with games to uncover bugs and 2) identify and determine if a given scenario exhibits a bug. While a definitive answer regarding the superior effectiveness of ABD techniques over manual testing in all aspects remains elusive, Dobles et al. [17] conducted a comparative study of automated and manual testing, focusing on effort and effectiveness. Their findings suggested that automated testing outperforms manual testing in defect detection. Conversely, automated tests require greater initial effort at the outset; however, they prove advantageous when testing needs to be executed multiple times, in contrast to manual testing approaches.

Iftikhar et al. [18] introduced an automated testing approach for functional testing of games using a model-based methodology. Evaluated through two platformer game case studies, the approach employed unified modeling language profiles, class diagrams, and state machines to generate and execute test cases automatically. This method automates key testing stages, including test case generation, test oracle creation, and test case execution. While it effectively detected faults, it requires users to have a software engineering background and create models before testing. The authors noted that the approach does not cover all paths and functionalities, potentially limiting its effectiveness for complex games.

Lovreto et al. [19] presented an innovative automated test script method for mobile games and evaluated its performance across 16 different mobile games. Despite their efforts to automate testing for mobile games, the functional test scripts were manually crafted, as highlighted by the authors. They also acknowledged several limitations associated with their approach, such as its inability to effectively address the unpredictable nature inherent in certain aspects of games.

Guglielmi et al. [20] introduced GamEpLay issue detector (GELID), which represents an innovative strategy for identifying anomalies in video games through the analysis of gameplay videos, aimed at assisting developers by offering insights for game improvement. The methodology behind GELID underwent empirical validation through the study of 604 video segments derived from 80 h of gameplay footage spanning three video games. The findings were varied: the processes of segmentation (step 1) and issue-based clustering (step 4) proved effective. However, the study revealed that the steps involving categorization (step 2) and context-based clustering (step 3) of

[1] [Online]. Available: https://github.com/EllieAzizi/AstroBugV1

segments require further refinement before they can be reliably implemented in practical applications.

Prasetya et al. [21] focused on navigation and exploration in video game playtesting, proposing an agent-based automated navigation framework for large game worlds. Their method reduces the search space by dividing walkable areas into interconnected geometric shapes, enabling agents to navigate using graph-based pathfinding. The framework employs rule-based and search-based techniques. However, it is limited in different environments and lacks generalizability. Exploratory search approaches like this struggle with adapting to various game systems due to constraints from predefined fitness functions and the challenge of vast state spaces.

Albaghajati and Ahmed [22] proposed a novel solution that leverages two agent-based genetic algorithms for video playtesting. The agents collaborate with one agent, buggy states generation agent, responsible for generating buggy states within the game, while the other agent, reachability agent, conducts an analysis of the game state using a representation based on Petri nets modeling. The authors themselves acknowledged that their approach, which relies on a rule-based logic, has limitations in identifying a root bug from which other related bugs may emerge. Consequently, their algorithm is unable to effectively pinpoint the primary bug that could potentially give rise to additional issues in the game.

Ferdous et al. [23] introduced a search-based test generating technique that creates a representation of the intended game behavior using an extended finite state machine. Abstract tests of the software model are generated through search-based algorithms. These abstract tests are then translated into sequences of actions, which are executed within the game being tested.

With a focus on striking a balance between the effectiveness and technical complexity of game testing, Zhao et al. [24] introduced a novel approach called LIT. This approach derives playing tactics from players' actual gameplay and utilizes them to automatically test the same game for potential bugs. LIT incorporates a library of rule-based inferences, which enables it to respond to various randomly generated game scenarios. However, it should be noted that the applicability of this framework may be limited when dealing with more complex games.

The absence of a unified platform or standardized benchmarks for ABD approaches has resulted in a fragmented body of literature and challenges in evaluating and comparing their performances, particularly in the context of exploration within ABD.

C. Integration of AI and Computer Vision in Game Testing

As modern video games continue to grow in complexity and the demand for quicker release cycles escalates, the conventional manual methods of bug detection have become increasingly time-consuming and expensive. AI-based approaches present a promising alternative, automating the bug detection process and delivering faster, more precise results.

In a study conducted by Chang et al. [25], a search approach based on rapidly exploring random tree algorithm called Reveal-More, was presented. Reveal-More combines automated game exploration with human gameplay, leading to enhanced coverage of different stages within the game. The technique contains two major phases. First, data collection, which depends on human effort. Second, amplification, which uses the first phase's game states to explore the game. The hybrid logic of Reveal-More was evaluated by conducting tests on two games [26], which resulted in improved coverage during the testing process. This approach places a greater emphasis on game exploration, aiming to highlight more moments of the game.

Nantes et al. [27] introduced a framework using AI and computer vision to streamline game testing. AI agents equipped with a GPU-vision debugger monitor user activity and provide an atomic regression test tool. The framework focuses on environment integrity inspection, particularly shadow rendering issues, aiming to expedite problem detection and resolution, thereby enhancing game quality and visual fidelity.

Ferguson et al. [28] used imitation learning to develop non player characters. They considered playstyle from observation of states and called it dynamic time wrapping imitation

Paduraru et al. [29] introduced the RiverGame tool, which aims to enhance the game testing process by considering various aspects, such as rendered output, sound, entity animation and movement, and performance. To facilitate analysis and testing, the researchers employed AI techniques and adopted the behavior-driven development methodology. RiverGame incorporates computer vision techniques, including image segmentation and object detection methods, to effectively detect and test visual bugs in gameplay.

Gudmundsson et al. [7] presented a novel framework for estimating the difficulty levels of *Match-3-Puzzle* games, utilizing a CNN trained on human-played data. The framework incorporates reward functions derived from human behaviors, which encompass atomic choices, to predict the difficulty levels of the games. By leveraging CNNs and analyzing human gameplay data, this approach offers a unique perspective on estimating the difficulty of *Match-3-Puzzle* games based on player behavior.

Lin et al. [30] investigated using online metadata to automatically identify gameplay videos showcasing bugs, providing an alternative bug information source for developers. Focusing on steam videos, they used a random forest classifier to rank videos by their likelihood of containing bugs. Their method showed a 43% increase in precision over basic keyword searches, tested on a dataset of 96 videos. Analysis of 1400 identified videos revealed a mean average precision at 10 and at 100 of 0.91, demonstrating the effectiveness of ABD in gameplay videos.

Keehl and Smith [31] introduced Monster Carlo 2, a machine testing approach that integrates AI with a tree search method. This innovative solution aims to automate the playtesting process for independent game developers. Building upon the Monster Carlo framework [32], Monster Carlo 2 utilizes the Monte Carlo tree search heuristic algorithm. By leveraging this approach, the system of Keehl and Smith [31], [32] enables independent game developers to conduct effective playtesting without requiring extensive knowledge of neural networks (NN) or significant computational resources.

D. Deep Learning and RL in Game Testing

Deep player behavior modeling was applied by Pfau et al. [33] to generate models capable of reproducing playtesting strategy. The primary focus of the testing was automated game difficulty balancing. However, the trained AI agents, which mimic the actions of individual players, can also be utilized for game exploration and detection of bugs and issues within the game.

Taesiri et al. [34] introduced a method using two deep convolutional neural networks (DCNN) to automate graphical bug playtesting. The system, operating in both local and cloud environments, detects corrupted frames by comparing them with healthy ones, achieving up to 90% accuracy. This method promises more efficient and reliable playtesting.

In another study, Taesiri et al. [35] proposed a search method using English text queries to retrieve gameplay videos without data labeling or training. Their dataset includes 26 954 videos from 1873 games sourced from Reddit. This approach, which achieved a 66.24% average recall rate, shows promise for future bug identification by locating objects within gameplay videos using text queries.

A novel classification method utilizing DCNNs was developed, employing the ShuffleNetV2 NN architecture by Ling et al. [37]. The performance accuracy of ShuffleNetV2 was compared with other well-known architectures, including ResNet [38], VGG [39], AlexNet [40], and MobileNet V2 [41]. Notably, the study highlighted the impact of different weight initialization techniques, comparing random initialization with pretrained weights from the ImageNet dataset. To train the classification algorithms, labeled datasets were used, consisting of various classes, such as normal images, stretched images, low-resolution images, missing textures, and placeholder textures. Their approach specifically aimed to detect graphical anomalies related to texture. This research demonstrates the efficacy of DCNN in identifying and classifying texture-related graphical anomalies.

RL was used by Xue [42] to test mobile apps automatically. Xue's approach utilized Q-learning to explore the graphical user interfaces of a diverse set of applications and generate behavior models. These models were subsequently used to optimize two objectives: minimizing the number of crashes and maximizing activity coverage. By leveraging RL, this method efficiently explores app interfaces and trains models that contribute to effective app testing.

Tufano et al. [43] introduced RELINE, an innovative method employing RL for the load testing of video games. RELINE's flexibility allows it to be applied across various games, utilizing distinct RL models and reward functions. Their initial study, conducted on two different systems, demonstrates the viability of this approach. With a reward function designed to incentivize the detection of artificial performance bugs, the RL agent adjusts its actions to continue gameplay, specifically targeting the identification of such bugs.

With the advancements in deep learning (DL) approaches for game testing, Wilkins et al. [44] conceptualized the problem of visual bug detection as an anomaly detection problem. In their method, they introduced a state-state Siamese network [45] specifically designed for this purpose. The algorithm is evaluated on a range of Atari games, showcasing its effectiveness in identifying anomalies. Notably, this anomaly detection solution requires significantly less data and training compared with other DL approaches, making it a promising direction for efficient and effective visual bug detection in games.

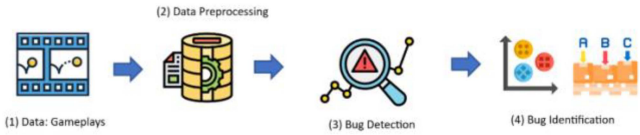

Fig. 2. AstroBug framework architecture.

Focusing on behavioral and perceptual bugs, World of Bugs (WOB) employs learning-based methods, utilizing the visual scenes seen by players [46]. It offers an open platform for ABD testing in 3-D game environments. An AI agent explores the game environment, capturing video frames as players would see them. To detect bugs within these frames, an autoencoder model is used, projecting input data into a lower dimension for bug identification. Overall, ten categories of perceptual bugs were identified and bug detection for certain categories of bugs was more accurate than for some others. The difference in the performance was due in some cases to the geometry on the back of the objects not getting rendered, making the model interpret the observations as normal.

It is crucial to remember that utilizing AI for bug detection marks a step toward improvement rather than an all-encompassing solution, as manual and ABD techniques continue to be employed in testing. Nonetheless, AI offers a swifter and more efficient avenue for game bug detection, with further enhancements yet to be realized. The above works mostly operate on game environment exploration than bug detection itself. Our work, however, detects bugs based on human played as well as AI played datasets and works around maximizing the detection of buggy frames.

III. Data

The overall architecture of the framework is illustrated in Fig. 2.

To effectively design and evaluate the proposed framework, it is crucial to consider the generalizability of the data specifications during the data requirement stage. This implies that both the framework itself and the data utilized for bug detection should adhere to a standardized approach that enables interoperability, accessibility, and scalability across different programming languages, platforms, and development environments. The tools and methodologies used for identifying, reporting, and fixing bugs should be applicable regardless of the specific technology stack or application domain. Within Section II, it is evident that most previous studies focused on utilizing specific data structures tailored to their respective methods. In contrast, our work adopts a more inclusive approach by considering gameplay data that can be collected from any designed games. The simplicity of this data structure facilitates its collection and subsequent analysis. Considering our approach as an anomaly

detection, the data requirements necessitate the use of time series. Time series data refer to a type of data where observations or measurements are recorded, organized, and indexed in a chronological order. In time series data, each data point is associated with a specific time stamp or a temporal indicator, indicating when the observation was made. Time series data are commonly used to analyze and understand patterns, trends, and relationships over time. This data type makes it possible to distinguish between buggy and bug-free frames. Also, it would be possible to detect bugs that otherwise would not be visible without considering previous frames. Some bugs are independent from previous scenes and can manifest randomly, while others are triggered by specific actions or events in previous frames. Consequently, we analyzed two distinct datasets: WOB [46] and Unity's *FPS microgame* [47]. Both games belong to the *FPS* genre, and their gameplay videos were recorded in the mp4 format.

A. *World of Bugs*

WOB is an open platform for testing automatic bug detection in 3-D games. The datasets created in this platform are divided into ten perceptual bug categories: texture missing, texture corruption, Z-fighting, Z-clipping, geometry corruption, screen tear, black screen, camera clipping, boundary hole, and geometry clipping [46]. WOB's data are obtained through the capture-and-collection process conducted by an AI agent. An example agent has been created using ML-agents package, facilitated by a Python application programming interface (API) extension of Unity's ML-agents API to demonstrate this capability; it navigates randomly while avoiding obstacles by choosing random points to move toward and deciding on actions, such as moving forward or turning. This exploratory approach, which intentionally lacks a defined goal for the agent, simplifies the development of models aimed at identifying bugs in video games. Subsequently, the dataset is partitioned into two distinct categories: training and testing. The training data comprise 300 000 observations and corresponding actions derived from a total of 60 episodes, each characterized by a bug-free gameplay, representing the normal state without any bugs present. On the other hand, the testing data encompass 500 000 observation–action examples that exhibit abnormal behavior by intentionally introducing bugs. The dataset is derived from the work of Wilkins and Stathis [46], who collected, preprocessed, and divided the dataset. To assess the performance of our framework on a more complex *FPS* game, we opted to utilize Unity's *FPS microgame*, a publicly accessible open-source game. To facilitate testing AstroBug, we deliberately injected perceptual bugs into the game. Specifically, we introduced three types of bugs: black screen, texture corruption, and boundary hole. These bugs were integrated into the game to simulate real-world scenarios. We chose to incorporate these three types of bugs out of the list of bug categories, randomly. The primary goal is to test if the framework would be able to detect anomalies in a more complex game compared with WOB's game environment. To conduct a comprehensive evaluation, we recorded and gathered gameplay footage from a diverse group of 25 players. On average, each gameplay recording spanned approximately 2 min, with players participating in multiple sessions. Consequently, we accumulated a substantial dataset comprising 300 min of gameplay recordings. Within this dataset, 70% (equivalent to 210 min) was dedicated to training, encompassing bug-free gameplay, while the remaining 30% (equivalent to 90 min) was reserved for testing, deliberately including gameplay segments that featured bugs.

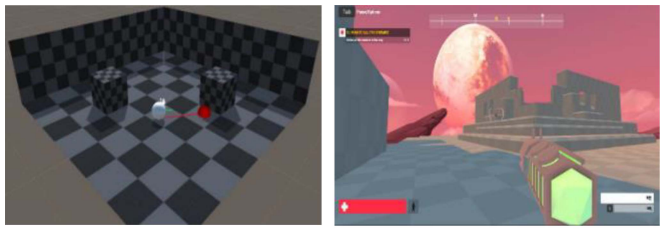

Fig. 3. Sample snapshot of (a) WOB dataset and (b) Unity *FPS microgame*.

B. *Unity* Microgame

Fig. 3 demonstrates an example of the game environment in WOB and Unity *FPS microgame*. The research was performed under the oversight of the institutional ethics review board. To ensure effective utilization of both datasets, we performed data preprocessing and prepared it for further training. Our data collector extracted 20 frames per second from the game plays. During the training process, the amount of video random access memory (VRAM) available on the GPU determines how many frames or images can be loaded into memory at once. To solve the VRAM capacity problem, we chose to load ten frames at a time into memory and process these frames as if they are a video clip having ten frames in each time slot, so the window size is 10. The window slide technique is a method employed to determine the composition of frames within each section of analysis. In this technique, the step size of the sliding window is set to 1, indicating that no frames are skipped during the analysis process. This choice of step size is based on the principle of inclusiveness, where each frame is meticulously considered in the evaluation. However, it is noteworthy that the step size of the window slide can be dynamically adjusted in response to the frequency with which bugs emerge over a specific time period and can vary with the bug frequency emergence. For instance, altering the step size to 2 would imply that one out of every two consecutive frames is skipped during analysis. This adaptation serves a dual purpose: conserving both memory resources and processing time if a bug takes more than two frames to be detected so that the model does not skip a bug. However, it is crucial to underscore that a step size of 1 remains indispensable in many gaming scenarios due to the inherent sensitivity of bug occurrence. For instance, consider the instance of a black screen bug that manifests and resolves within a single frame.

In such cases, a step size of 1 ensures that no potential bug is overlooked, as even a momentary aberration can have a significant impact on the gaming experience. Constructing a video clip of ten frames makes it easier and faster to train and

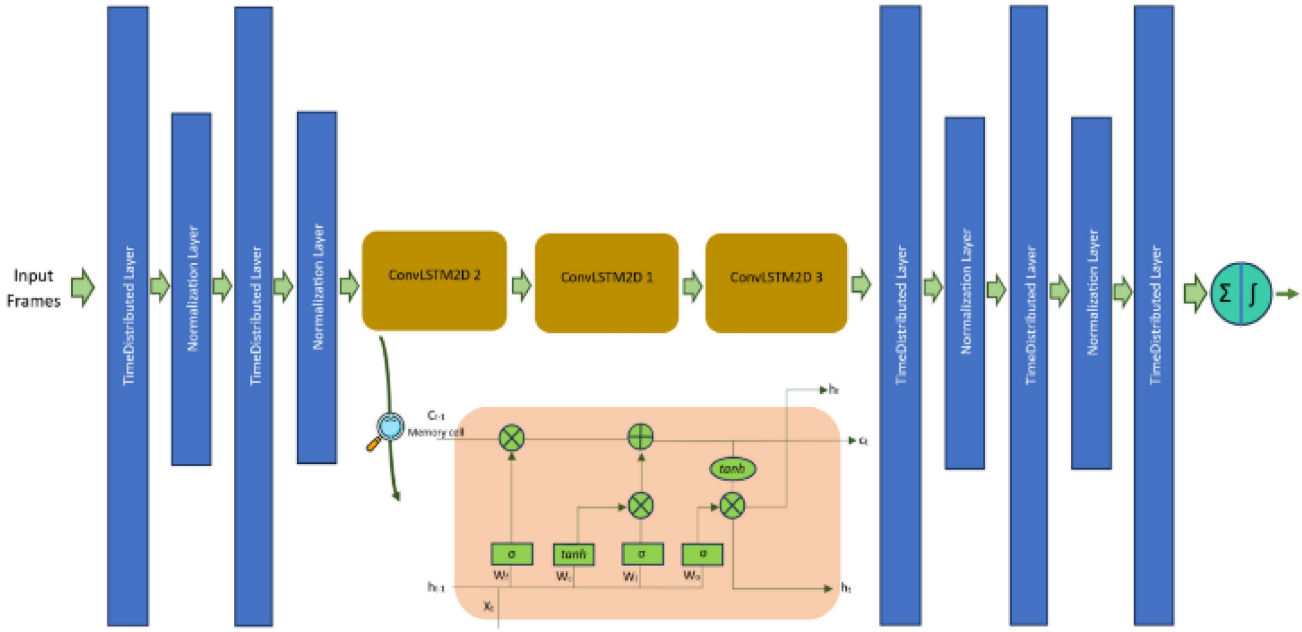

Fig. 4. AstroBug's model architecture.

evaluate the model. The determination of the window size is contingent upon factors, such as time considerations, memory constraints, available resources, accuracy targets, and associated costs. While identifying the optimal window size that yields the highest accuracy is an ongoing exploration, Jaén-Vargas et al. [48] suggested that the most effective window size lies within the range of 20–25 frames. Notably, larger window sizes exhibit marginal variations in accuracy, whereas smaller sizes exhibit a decline in accuracy. Considering these findings, an avenue to potentially enhance accuracy involves enlarging the window size. This adjustment would account for more extended data dependencies, likely leading to more refined outcomes. However, it is imperative to note that in our specific dataset, the temporal dependencies between frames and their associated bugs are relatively short lived. Moreover, practical hardware limitations have played a role in influencing the decision-making process. Ultimately, striking a balance between window size and accuracy is a nuanced endeavor. While longer window sizes have the potential to incorporate more comprehensive bug-related context, the peculiarities of our data suggest that this advantage may not be fully realized due to the transient nature of frame dependencies. In the context of video game testing, however, since testers employ various methods distinct from ours, it cannot be definitively stated that a window size of 10 is commonly adopted across the board but we chose to set to 10 considering our limitations.

IV. Model

In the bug detection part of the framework, to observe the sequence of actions, we have used LSTM architecture in our framework. LSTM networks are a type of recurrent NN [49], which excel at capturing long-term dependencies and patterns in sequential data. In LSTM, earlier inputs can affect later outputs in a sequence since these models can selectively forget or remember information over time. In our work, the model we came up with is composed of two consecutive TimeDistributed layers [50], followed by three Convolutional LSTM layers and three additional TimeDistributed layers. Fig. 4 illustrates the comprehensive architecture of the proposed model, drawing inspiration from Chong and Tay [51].

The model processes preprocessed gameplay frames by passing them through the initial TimeDistributed layer. The TimeDistributed layer is typically used with sequential data and acts as a wrapper, allowing a specific operation to be applied independently to each time step of a sequence. This wrapper facilitates the modeling of intricate dependencies and provides flexibility in handling sequential data within LSTM networks. In our specific scenario, we group consecutive frames into window slots of 10, which are encapsulated within a single wrapper. The model generates predictions based on these 10-frame windows and continues this pattern for subsequent frames in the sequence. This approach ensures that the model maintains a continuous and contextual understanding of the gameplay frames, enabling it to make accurate predictions and effectively capture the temporal dynamics present in the data. Each of the TimeDistributed layers is followed by a normalizing layer [86]. The normalization layer transforms the input features into a consistent and comparable range, preventing certain features, which are the attributes within the data that the DNN autonomously identifies, from dominating the learning process due to their larger magnitude. The last TimeDistributed layer is followed by a sigmoid activation function [52]. The sigmoid function takes any real-valued input and maps it to a value between 0 and 1. We employed the mean squared error [52] as the loss function to leverage its advantages, such as penalizing larger errors more heavily while maintaining simplicity. Loss function measures the difference between the predicted output of a model and the true target values [53].

Finally, we utilized the Adam optimizer [54] with a learning rate set to 0.0001 as the optimizer. Optimization algorithms determine how the model's parameters are adjusted in each iteration to gradually improve the model's performance.

Once the model has been trained using the provided frames, we proceeded to evaluate its performance on two specific datasets mentioned earlier. To conduct the testing phase, the testing data underwent the same preprocessing steps as the training data. Our testing approach involved comparing sets of ten frames from the test data to the normal patterns learned by the model, both at the individual frame level and collectively as sequences of ten frames. The objective was to measure the dissimilarities between the test frames and the learned patterns, which would indicate the extent to which the frames deviated from normalcy. To quantify these differences, we employed a statistical method known as a regularity score (RS) metric [55]. Specifically, we calculated the probability density of the data points, providing a distance metric to assess the abnormality of the frames. The RS, based on this statistical approach, served as a measure of the anomaly detection capability of our model. The RS serves as a tool to identify and prioritize potentially anomalous or unusual instances in a dataset. By setting appropriate thresholds, it can help distinguish between normal and abnormal patterns. Higher RSs indicate greater regularity or similarity to the expected patterns, whereas lower scores indicate greater deviation or anomalousness. The RS results based on the corresponding frames were visually presented in the form of diagrams. The diagrams featured the frame number on the x-axis and the RS values on the y-axis. This graphical representation allowed for a clear visualization of the RS trends and variations across the frames.

V. Bug Identification

Knowing the bug category provides game developers with valuable insights and assistance in multiple aspects of game development. Bug identification allows developers to fix bugs efficiently by streamlining the debugging process, which results in time saving and ensuring that the appropriate resources are allocated to resolve specific types of issues. Developers can leverage their understanding of bug categories to employ effective debugging techniques, use relevant tools, and address the root causes more efficiently. This information can be helpful in identifying potential areas of improvement in their codebase, design, or development processes and communicate more effectively with team members. Developers can also benefit from prioritizing bug fixes by knowing the bug categories since not all bugs have the same impact on gameplay or user experience. Prioritization ensures that critical issues are addressed promptly, reducing the risk of detrimental effects on the game's reputation and player satisfaction.

We identified similarities in the patterns exhibited by the RS diagrams corresponding to a particular bug category in the WOB dataset. In other words, the RS diagrams representing a bug category reflected a similar RS diagram pattern. Recognizing the significance of these similarities, our objective was to develop a methodology to categorize the detected anomalies and effectively identify the bug categories they represent. To accomplish this goal, we employed an unsupervised clustering algorithm known as DBSCAN. By leveraging the algorithm's ability to identify regions of high density within RS diagrams, we could effectively segregate and classify the RS diagrams associated with bug categories. Although using the frames where a potential bug might have occurred is an alternative method to pinpoint the bug type, leading to potentially more accurate predictions, it still necessitates human involvement for labeling the data where the bug appeared. Moreover, automating the bug identification process with the original frames would require a pretrained model based on previously labeled data. Alternatively, even if we opt for unsupervised learning to cluster anomalies found in the original frames, the developer would need prior knowledge of the possible bug types, which contradicts the primary objective of this research: to proceed without such information. A more autonomous approach would involve employing unsupervised learning with RS diagrams. Unlike traditional clustering algorithms, such as K-means, DBSCAN does not require the number of clusters to be predefined and can discover clusters of arbitrary shapes. This is useful in our case since we wanted to test if the algorithm would be able to distinguish the ten bug categories and find ten clusters or not. This flexibility allows DBSCAN to discover clusters of arbitrary shapes based on the density of the data points. By allowing DBSCAN to adaptively determine the number of clusters based on the data distribution, we could assess its capability to uncover the inherent structure and differentiate the bug categories without imposing any prior assumptions or constraints. While DBSCAN does not require a predefined number of clusters, it still relies on two key hyperparameters: the minimum number of points and the distance threshold. The minimum number of points determines the threshold for forming dense neighborhoods within a cluster in DBSCAN. In our specific case, we set this parameter to 9 based on our observations within the WOB test dataset. Typically, each bug category in the dataset had nine gameplays associated with it, resulting in nine corresponding diagrams, except for the boundary hole category, which had 29 gameplays. Consequently, we expect the clusters generated by DBSCAN to contain approximately nine samples, aligning with the average number of diagrams per bug category. For the LSTM model utilized in our study, it is essential to clarify that the distribution of videos is primarily affected during the testing phase rather than the training phase. This distinction is critical because, during training, an imbalanced dataset might indeed introduce bias toward more frequently represented categories, potentially skewing the model's learning process. However, since the "boundary hole" videos are part of the testing set, their presence in higher numbers does not directly impact the model's learning but rather serves to evaluate the model's generalization capability across different scenarios and categories. In the context of clustering, where the DBSCAN algorithm is employed to categorize anomalies, the distribution of videos indeed plays a more pivotal role. The "boundary hole" category, being denser due to its higher number of samples, is more likely to form a distinct cluster. This is attributable to DBSCAN's design, which favors areas of higher density. Conversely, categories with fewer

videos face a different set of challenges, as their ability to form identifiable clusters depends on their internal sample density and the spatial distribution relative to other categories. Here, the process of parameter tuning, particularly selecting appropriate values for DBSCAN's epsilon and distance thresholding, is crucial.

Furthermore, the distance threshold establishes the maximum allowable distance between two data points to be considered neighbors. In our case, we set the distance threshold to 0.3. This choice was motivated by the fact that the diagrams exhibit subtle variations in appearance, and we wanted the algorithm to be particularly sensitive to these small differences in patterns. By setting a relatively low distance threshold, we aimed to capture even the subtlest dissimilarities between data points, enabling the clustering algorithm to effectively distinguish between closely related patterns. The distance threshold parameter plays a significant role in determining the size and shape of the clusters identified by DBSCAN. A smaller Epsilon value results in tighter and more compact clusters, as points must be in close proximity to be considered neighbors. Conversely, a larger Epsilon value allows for the inclusion of points from a broader neighborhood, potentially capturing larger clusters or groups of points that are more spread out. We made the decision to utilize grayscale 224 $\times$ 224 images in our approach since the color of the diagrams does not significantly contribute additional information to the training, evaluation, and overall results of the model. The data are presented in the form of diagrams, where, notably, the color of a line is uniform across all diagrams and can be adjusted according to our preferences. This consistency in color means that it does not serve as a distinguishing feature or convey any additional information within the data visualization. Essentially, the color choice is arbitrary and does not contribute meaningful insights or highlights specific aspects of the data being represented. Therefore, when analyzing or interpreting these diagrams, the color should not be considered as a factor that carries any significant information or relevance.

VI. Evaluations

In the WOB dataset, we generated RS diagrams for each of the ten bug categories from video gameplays. During our analysis, we observed the behavior of the RS diagram. We noticed that the RS diagram exhibits a consistent pattern, whereby the RS values remain relatively stable for frames without any detected bugs. However, once a bug is encountered within a frame, there is a noticeable drop in the RS value within the corresponding region. By examining the RS diagram, we can visually discern the areas where bugs manifest in the gameplay. The distinct drop in the RS values within these regions indicates the presence of anomalies or irregularities associated with the bugs. This characteristic of the RS diagram provides valuable insight into the locations and extent of bugs within the gameplay sequence.

To provide a visual representation of the process, we present Fig. 5, which offers a specific example showcasing the output related to the boundary hole bug category, accompanied by its corresponding frame. In Fig. 5, we observe a graphical depiction of the results obtained from the analysis in which it becomes evident that the RS value experiences a substantial drop, reaching as low as 0.09 in frame 268. The *y*-axis denotes RS and *x*-axis denotes the frame number in RS diagram. This significant decrease in the RS value is directly attributed to the presence of the boundary hole bug within the video sequence.

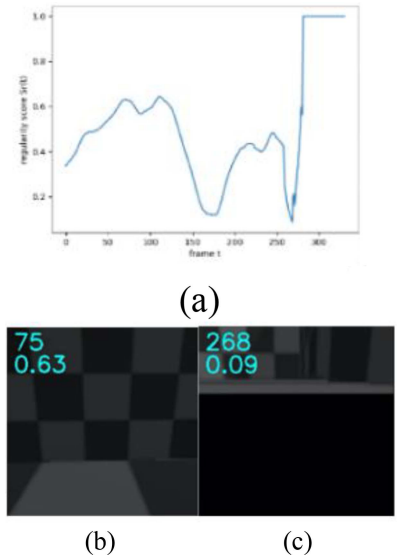

Fig. 5. Example of the framework's output of a boundary hole bug in the WOB game. (b) and (c) represent the time and the RS of the frame. (a) RS diagram, (b) a normal frame, and (c) a buggy frame.

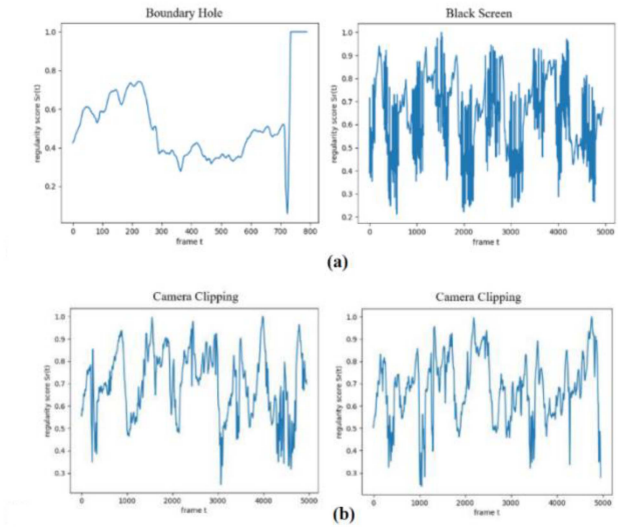

Fig. 6. Illustration of RS diagram pattern of three bug categories. *X*-axis in diagrams represents frame in time *t* and *y*-axis represents RSs in time *t*.

Our analysis of the WOB dataset revealed distinct patterns in the output corresponding to the ten different bug categories. Each bug category exhibited unique characteristics, showcasing variations in their respective outputs. Notably, we observed that the outputs within a specific bug category tended to exhibit similarities, indicating consistent patterns across instances of that particular bug category.

To visually illustrate this phenomenon, we present Fig. 6(a), which showcases the results obtained from testing on two different frames, each containing a different type of bug: black screen and boundary hole. The figure provides a side-by-side

TABLE I
RS THRESHOLD VALUE CORRESPONDING TO BUG CATEGORIES IN WOB

Bug category	Threshold value
Texture missing	0.5
Texture corruption	0.22
Z-fighting	0.45
Z-clipping	0.35
Geometry corruption	0.45
Screen tear	0.37
Black screen	0.48
Camera clipping	0.38
Boundary hole	0.18
Geometry clipping	0.4

comparison of the output patterns for these two distinct bug categories. In contrast to the observations made in Fig. 6(a), Fig. 6(b) presents an intriguing scenario where the output patterns in the diagrams exhibit striking similarities. In this case, both diagrams correspond to instances of the camera clipping bug. The visual representation in Fig. 6(b) provides a clear depiction of the shared patterns and characteristics between these two instances of the camera clipping bug. By closely examining the diagrams, we can observe the consistency in the irregularities and anomalies associated with this particular bug category. In addition, a closer examination of Fig. 6 provides valuable insights into the impact of various software bugs on the gaming experience. It illustrates that certain bugs can have more severe consequences than others. For instance, when a boundary bug manifests, it results in a critical failure where the game abruptly ceases to function. Players are left facing a blank, black screen, which signifies an unexpected and complete halt of the game, essentially forcing them to restart or exit the game entirely. This type of bug disrupts the gaming experience significantly, as it not only interrupts gameplay but also potentially leads to loss of progress, thereby frustrating players. On the other hand, other types of bugs, camera clipping and black screen in Fig. 6, contrast this with less severe issues. These are described as less impactful on the overall gameplay continuity. Despite the potential for these bugs to cause visual glitches, such as parts of the game world disappearing or the camera moving through solid objects unexpectedly, they do not prevent the game from continuing. Players might experience momentary confusion or distraction, but they can generally proceed with their gameplay without needing to restart or quit the game. This distinction highlights the varying degrees of disruption caused by different types of bugs, underlining the importance of prioritizing bug fixes based on their impact on the player's experience.

The analysis of RS thresholds for different bug categories in diagrams revealed significant variation. RS thresholds indicate bug severity within specific frames, with each bug category having a unique threshold. Understanding these thresholds is vital for precise bug detection and categorization in gameplay. Setting appropriate thresholds helps differentiate normal gameplay from bug-induced anomalies, enhancing bug resolution for game developers and testers. Table I summarizes the observed threshold values for each bug category. In the WOB dataset, each bug category is clearly labeled and distinguishable from others, making it possible to identify unique patterns in RS diagrams. By closely analyzing these diagrams, we can observe the differences in the RS threshold for each bug category. These thresholds indicate how notably a bug appears within a specific frame. The initial step involves examining random sample diagrams alongside the corresponding gameplay videos. We then considered the frames where the bugs started to occur and recorded their RS values. Calculating the average of these RS values provided us with the threshold levels for each bug category.

Following the training phase of our model using the bug-free data from the Unity *FPS* game dataset, we proceeded to evaluate its performance on gameplays containing buggy frames. During testing, we made a notable observation: while the model successfully detected anomalies within the gameplays, the resulting diagrams did not exhibit a consistent pattern compared with the WOB dataset. The lack of a consistent pattern in the diagrams can be attributed to three main factors. First, the buggy videos in the Unity *FPS* dataset encompassed a diverse range of bugs, with each gameplay containing distinct bug types. Consequently, the irregularities detected by the model varied significantly from one gameplay to another. This variability in bug types led to the absence of a uniform pattern across the diagrams. As a result, the RS diagrams generated from the WOB testing dataset exhibited clear patterns and distinct irregularities corresponding to specific bug categories.

To assess the performance of the clustering model implemented for bug identification using RS diagrams in the WOB dataset, we employed the homogeneity score metric [56]. The homogeneity score is a clustering evaluation metric that quantifies the extent to which each cluster comprises only samples belonging to a single category or bug type. While other metrics, such as silhouette and V-measure, are valuable for evaluating clustering algorithms in general, we prioritized the homogeneity score in this study due to its relevance to the specific context and objectives. Homogeneity score captures the degree of homogeneity within each cluster, considering the consistency of bug category assignments within the clusters. It focuses on the quality and purity of the clustering results and serves as a critical metric for evaluating how well a clustering solution groups together data points that are members of a single class, which aligns properly with our evaluation intentions. The evaluation of the clustering algorithm on the RS diagrams yielded promising results, with a homogeneity score of 0.74. This score indicates a substantial level of homogeneity within the clusters, signifying that the algorithm successfully grouped similar diagrams together based on their patterns and characteristics. However, upon further analysis, we observed that two specific classes of diagrams, namely, camera clipping and geometry clipping, exhibited considerable similarity in their output patterns. This similarity in the diagrams led to potential confusion and overlapping between these two bug categories. To address this issue and improve the clustering performance, we decided to exclude these two classes from the analysis. By removing the camera clipping and geometry clipping classes from the dataset, we aimed to mitigate the potential misclassification or overlap between these categories.

Fig. 7. Demonstration of *Hogwarts* game world.

This refinement resulted in an enhanced homogeneity score of 0.85, signifying a higher level of consistency and accuracy in clustering similar diagrams together. When examining the effect of grouping two categories together versus treating them as distinct entities, we found a negligible difference in outcomes. This observation stems from the clustering algorithm's (specifically DBSCAN's) performance, which struggled to effectively separate these groups when both were included in the analysis. Given that DBSCAN does not require predefined cluster labels, the decision to merge or maintain these categories as separate does not fundamentally alter the clustering process. Essentially, whether the data from both categories are combined or presented separately, DBSCAN's approach remains unchanged, as it inherently clusters data without regard to specific labels. The notion of forcibly merging these categories proves inconsequential within the context of DBSCAN's methodology, which autonomously determines clusters based on data density, independent of any preassigned categorization.

VII. RL Agent

Collecting data from WOB and Unity *Microgame* games is generally not overly challenging, considering their scale and environment. However, a lingering question remains: Can AstroBug effectively test games with minimal human intervention and detect perceptual bugs in modern, complex video games as effectively as it does for WOB and the Unity *FPS microgame*? To address this question and evaluate the framework's capabilities more accurately, we made significant refinements and introduced two key changes to AstroBug.

First, we selected a complex *RPG* game known as *Hogwarts*, inspired by the beloved Harry Potter series [57]. *Hogwarts* offers a vast open sandbox environment. Within this game, players can freely explore the intricately designed 3-D world, engaging in various activities, such as walking, jumping, and interacting with game features, including maps, riding brooms, and engaging in combat against enemies. The *Hogwarts*' game world is represented in Fig. 7.

Second, to be able to collect train and test data for AstroBug automatically, we created an AI agent for *Hogwarts*. For this purpose, we used Unity ML-agent package and trained the agents to explore the game based on a reward–punishment approach.

The primary objective of the developed agent is to navigate and explore the game environment while taking appropriate actions. While incorporating reward mechanisms based on agent performance, such as defeating enemies, could potentially result in more realistic gameplay experiences, we intentionally opted for a simpler design, which is world exploration, for three key reasons. First, our primary motivation for creating an AI agent is to collect a comprehensive dataset. It is crucial for developers to thoroughly examine and analyze every interaction and aspect of their games to ensure that they are free from bugs and issues. In the context of perceptual bug finding, our aim is to have the AI agent that explore the game environment to identify any visual glitches or anomalies by the framework. By ensuring broad coverage of the agent's environment, we align with the intentions of AstroBug in detecting and addressing such bugs. Second, at this particular stage of development, our main focus is on the performance and capabilities of the AI model itself rather than intricate and complex gameplay scenarios. By simplifying the agent's objectives and prioritizing the refinement and evaluation of the underlying model, we can better assess its effectiveness and potential for future enhancements. Third, game exploration makes the approach and framework more generalizable and adoptable. By integrating game exploration over exploitation, we extend the capabilities of our framework beyond specific game genres or scenarios.

The agent has a range of actions at its disposal to navigate the enchanting environment. These actions include moving left, right, forward, and back, allowing the agent to explore different directions within the 3-D space. In addition, the agent can jump to overcome obstacles, mount a broom to engage in aerial traversal, and wield the power to attack adversaries it encounters. To shape the learning process and incentivize desirable behaviors, we have defined rewarding mechanisms based on the player's survival. If the agent successfully stays alive, it receives positive rewards that reinforce its ability to navigate the challenges and complexities of the *Hogwarts* world. These rewards encourage the agent to learn effective strategies for survival, such as avoiding hazards. Conversely, when the player's character dies, negative rewards are applied to discourage actions or behaviors that lead to unfavorable outcomes. By associating penalties with death, the agent learns to avoid actions or situations that put it at risk and instead seeks actions that promote longevity and progress within the game. To train the agent to effectively navigate the intricacies of *Hogwarts* and master its various mechanisms, we employed the powerful deep Q-learning algorithm [58]. Deep Q-learning enables the agent to estimate the values of state–action pairs, known as Q-values, which represent the expected cumulative rewards for specific actions in specific states.

After completing the training phase, we proceeded to collect the dataset from the agent's gameplay. In addition, to enhance the training process, we included two gameplay recordings from human players that were confirmed to be free of any bugs. Our dataset consists of 41 min of gameplay, and while a larger dataset might have strengthened our findings, practical constraints limited our ability to process and analyze data to ensure that it was free of bugs. These recordings served as valuable testing data for the model. While this approach makes the process not entirely automated, it addresses the issue of

TABLE II
SUMMARIZATION OF THE RESULTS ACHIEVED FROM TESTING *HOGWARTS* GAME WITH DIFFERENT VALUES OF RS THRESHOLD VALUES

RS threshold value	Total number of returned frames (/22 000 frames)
0.4	12 060
0.3	10 050
0.2	6007
0.1	1503

anomaly detection. By incorporating bug-free frames observed during human gameplay, we aimed to minimize any potential bias toward anomalies that might arise from data collected solely from the RL agent. Although this data collection process does require some time and effort, it is considered reasonable in the context of game development. It is common for developers to review and observe gameplay sessions before publishing a game. From RL agent, we obtained dataset comprising 250 000 frames of gameplay. All these data were used to train the model. Recognizing the importance of covering a broad range of gameplay scenarios for an effective bug detection system, the authors acknowledge the limitations posed by sourcing footage from a single player, as it may not fully capture the diversity of gameplay experiences and potential bugs. The variability in player strategies and interactions further complicates achieving a comprehensive dataset. To overcome these challenges and expand our analysis, we incorporated RL simulations with human gameplay footage. This hybrid approach aims to create a dataset that more accurately represents the wide array of gameplay scenarios, enhancing the effectiveness and comprehensiveness of our automatic bug detection system. On the other hand, to further validate the effectiveness of our model and assess its performance in real-world scenarios, we conducted testing using human gameplay recordings. By incorporating gameplay data from human players, we sought to simulate and evaluate the model's performance in capturing real-world and actual gameplay dynamics. We also tested the model based on the human gameplays to somehow represent and test real-world and actual gameplays. By subjecting the model to this additional testing based on human gameplay, we aimed to assess its generalizability and robustness across different playstyles and decision-making patterns. This evaluation process allowed us to gauge the model's performance against real-world gameplay expectations and potential deviations from the agent-generated data.

Unlike the approach of using RS diagrams to test the initial design of the framework, we evaluated AstroBug's performance in the *Hogwarts* game by employing a range of threshold values from 0 to 0.4, with a step size of 0.1. At each iteration, we identified the frames that fell within these thresholds and examined whether any bugs were present in those frames. The findings from this analysis are summarized in Table II.

Because of the high number of buggy frames returned from the model, we calculated the false positive (FP) and true positive (TP) rate for only the threshold 0.1. FP occurs in situations where a test or evaluation incorrectly identifies a condition or outcome as positive when it is actually negative or absent. In our specific scenario, FP refers to cases where the returned frames are identified as buggy frames, indicating the presence of bugs or anomalies, when in reality, the frames are not actually buggy. FP occurs when our bug detection system incorrectly flags frames as problematic, leading to a higher number of false alarms or unnecessary notifications for developers. TP refers to cases where a test or evaluation correctly identifies a condition or outcome as positive when it is indeed present. It represents the successful detection or identification of a desired or expected outcome. In our specific scenario, TP refers to the cases where the bug detection system correctly identifies frames as buggy frames, accurately flagging and reporting actual issues present in the game. The FP gained from evaluating the RS values of threshold 0.1 in *Hogwarts* is 211 and TP is 1292.

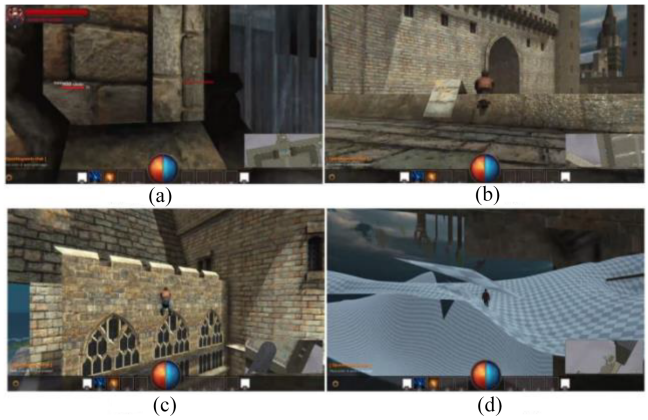

Fig. 8. Bug category samples in the *Hogwarts* game. (a) Camera instability. (b) Missing collider. (c) Camera clipping. (d) Geometry corruption.

From the FP and TP results, it can be concluded that AstroBug could locate perceptual and behavioral bugs effectively since, out of 100 buggy reported frames, on average, 86 of them were actually buggy leaving 14 frames as nonbuggy. Hence, we can estimate the precision of the model. Precision measures the proportion of correctly predicted positive instances out of all instances predicted as positive by the model and is mathematically represented in the following equation:

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \tag{1}$$

Considering (1), it yields to a precision of 86% for the framework given the mentioned conditions. Refer to Fig. 8 for instances of the detected bugs. The WOB dataset exclusively contains perceptual bugs, with no instances of behavioral bugs present in the dataset. We did not observe any specific behavioral bugs in the second dataset as well. However, the framework was able to detect any anomalies including behavioral bugs in *Hogwarts* game (A missing collider example is provided in Fig. 8). This is because of the nature of the game and data and not the performance of AstroBug.

By conducting a more in-depth analysis of the patterns and the number of frames reported as buggy in *Hogwarts* game, and comparing them with the number of frames reported from the WOB and Unity *microgame*, we observed a significant increase

in the number of frames flagged as buggy in our target game. This observation led us to an important realization: the high number of returned buggy frames was primarily due to the game itself containing a substantial number of visual glitches and behavioral bugs. Our analysis revealed that the target game exhibited a higher frequency of visual anomalies, such as graphical corruptions, rendering inconsistencies, or animation glitches. These perceptual bugs contributed to the increased number of frames being flagged as problematic by our bug detection system.

VIII. Conclusion

AstroBug is a framework designed to assist in bug detection and testing in game development. The framework analyzes the frames of gameplay and identifies instances that deviate from the expected behavior, signaling the presence of bugs, using ConvLSTM networks. To evaluate the effectiveness of our framework, we conducted experiments on two FPS games, WOB and Unity *FPS microgame*. The dataset used for the testing and training in the WOB originates from work specifically conducted within the WOB framework. On the other hand, the Unity *FPS microgame* dataset is derived from a collection curated by our team. The results showcased the framework's ability to accurately identify perceptual bugs, validating its effectiveness in bug detection.

Furthermore, we extended the capabilities of the framework by integrating an RL agent. This RL agent autonomously gathers datasets, reducing the reliance on human players to manually collect data and browse through the games. The gameplay videos of *Hogwarts* game that form a part of our dataset are also sourced from our own collection efforts. This collection is unique as it includes not only human gameplay footage but also gameplay generated by RL agents. By leveraging RL techniques, we enhance the efficiency and automation of data collection, making the bug detection process more streamlined and less dependent on human intervention. This enhancement was implemented and evaluated in the context of an *RPG*.

The outcomes of our experiments provide the framework's effectiveness in detecting bugs and showcasing its potential to improve the bug detection process in game development. Our framework demonstrates promising capabilities for identifying and addressing perceptual bugs, ultimately enhancing the overall quality and player experience in video games.

A. Limitations and Future Work

AstroBug has demonstrated promising outcomes in the detection of perceptual bugs at its current stage. However, to ensure broader applicability and ease of adoption by game developers and companies, it is crucial to develop a more generalizable interface. Although AstroBug is currently in the research stage, our vision for the future entails creating a user-friendly interface that can be seamlessly integrated into games for extensive testing. By building a user-friendly interface, we aim to streamline the adoption process and encourage widespread utilization of AstroBug within the game development community.

While the bug identification process for the WOB dataset has shown satisfactory performance, its applicability in real-world scenarios may be limited. Unlike the controlled environment of the WOB dataset, games, such as Unity *FPS microgame,* can feature mixed types of bugs within a single gameplay. Distinguishing between these different bug types solely based on threshold values may not always yield accurate results. To address this challenge and improve bug identification accuracy, a more effective approach involves clustering all identified anomalies based on the frame itself. By locating the specific frame where a bug is detected, developers can gain deeper insights into the nature and characteristics of the bug and prioritize them based on the frequency. This allows for a more precise categorization of the bug, as the frame's content provides valuable contextual information. Due to the complexity of the frequency of the buggy frames detected in *Hogwarts*, categorizing behavioral bug types in *Hogwarts* game presents a significant challenge, if not an impractical endeavor. Behavioral bugs still require further exploration compared with perceptual bugs, and it remains to be seen if the framework can effectively differentiate between them.

Unsupervised learning algorithms can have comparison baselines, but without labeled data, it becomes challenging to directly compare outcomes across different types of games, due to the varying complexity and nature of these genres. While the labeled WOB dataset simplified evaluations, making it easier to understand, our efforts to define evaluation metrics for the unlabeled data of the *Hogwarts* game encountered difficulties. In addition, due to the absence of detailed previous results of bug detection in the WOB dataset, we were not able to precisely compare our findings with previous efforts. We suggest that the best approach is to apply the AstroBug methodology to specific games individually to assess its effectiveness. While we conducted a qualitative evaluation of AstroBug, it is essential to quantitatively assess the effectiveness of the framework. One such evaluation metric is the false positivity rate, which measures the instances where the model incorrectly predicts a frame as being buggy when, in reality, it is not. Other possible metrics can be area under the curve (AUC) and recall values. To perform this quantitative evaluation, we require labeled testing data frames, against which we can compare the predictions made by the model. This allows us to determine the accuracy and reliability of the framework in identifying bugs. Quantitative testing provides valuable insights into the performance of AstroBug, enabling us to assess its precision and potential limitations. However, it is important to note that quantitative testing is a time-consuming process and can only be conducted once due to the effort required in labeling the testing data frames. Nevertheless, the insights gained from this evaluation contribute to a more comprehensive understanding of the framework's effectiveness and aid in further refining its performance. One of the methods that can be adopted in future to provide comparison baselines is the use of autoencoders in which the reconstruction error can serve as a baseline of comparison in locating anomalies.

Understanding the significance of window size in the accuracy of our analysis, we believe that a comprehensive investigation into varying window sizes, particularly within the 20–25 frame range recommended by Jaén-Vargas et al. [48], could yield valuable insights and potentially enhance the robustness of our results. To this end, the authors have acknowledged the

exploration of different window sizes and the importance of finding a balance between window size and accuracy as an important avenue for future research.

The selection of DBSCAN for our study was guided by several considerations specific to our research objectives and the nature of our data. DBSCAN can identify clusters of arbitrary shape and does not need the specification of the number of clusters, which is advantageous in scenarios where the structure of the data is unknown or complex. These characteristics aligned closely with the requirements of our project, where the primary goal was to categorize diagrams with irregular patterns. The authors acknowledge the value that a comparative analysis of clustering algorithms (e.g., NN or HDBSCAN) could bring to the research, offering insights into the relative performance and suitability of different methods for similar tasks.

In addition to focusing on perceptual and behavioral bugs, we recognize the importance of addressing logical bugs, which are also prevalent in games and can be considered as anomalies. Logical bugs, arising from flawed game rules or erroneous implementation, can impact the gameplay experience and hinder the overall quality of the game. To improve AstroBug's capabilities, we aim to incorporate logical bug detection into the framework. By expanding its bug detection algorithms and techniques, we can enable AstroBug to identify and flag logical anomalies, helping developers pinpoint and rectify logic-related issues that affect the game's functionality, progression, or overall gameplay experience.

As an extension, enhancing the RL agent to mimic player actions more closely offers a baseline of comparison in the future. This development, despite potentially limiting generalizability, opens up opportunities for in-depth analyses of the agent's performance across different types of video games, contributing to the overall validity of the collected dataset.

As AstroBug evolves, it will continue to be refined based on the feedback from game developers, testers, and players. The goal is to create a framework that not only excels in bug detection but also supports the iterative development process, allowing developers to iterate and improve their games more effectively.

Acknowledgment

The authors would like to thank Daniel McCormick for his invaluable assistance in preparing the FPS dataset used in this research.

References

[1] S. Ariyurek, A. Betin-Can, and E. Surer, "Automated video game testing using synthetic and humanlike agents," *IEEE Trans. Games*, vol. 13, no. 1, pp. 50–67, Mar. 2021, doi: 10.1109/TG.2019.2947597.

[2] A. Rollings and E. Adams, *Andrew Rollings and Ernest Adams on Game Design*. Indianapolis, IN, USA: New Riders, 2003.

[3] A. Albaghajati and M. Ahmed, "Video game automated testing approaches: An assessment framework," *IEEE Trans. Games*, vol. 15, no. 1, pp. 81–94, Mar. 2023, doi: 10.1109/TG.2020.3032796.

[4] B. Wilkins and K. Stathis, "Learning to identify perceptual bugs in 3D video games," Feb. 2022, *arXiv:2202.12884*.

[5] J. Bergdahl, C. Gordillo, K. Tollmar, and L. Gisslén, "Augmenting automated game testing with deep reinforcement learning," in *Proc. IEEE Conf. Games*, 2020, pp. 600–603, doi: 10.1109/CoG47356.2020.9231552.

[6] A. Nantes, R. Brown, and F. Maire, "Neural network-based detection of virtual environment anomalies," *Neural Comput. Appl.*, vol. 23, no. 6, pp. 1711–1728, Nov. 2013, doi: 10.1007/s00521-012-1132-x.

[7] S. F. Gudmundsson et al., "Human-like playtesting with deep learning," in *Proc. IEEE Conf. Comput. Intell. Games*, 2018, pp. 1–8, doi: 10.1109/CIG.2018.8490442.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

[9] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 15:1–15:58, Jul. 2009, doi: 10.1145/1541880.1541882.

[10] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discov. Data Mining*, 1996, pp. 226–231.

[11] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, pp. 237–285, May 1996, doi: 10.1613/jair.301.

[12] E. Azizi and L. Zaman, "Automatic bug detection in games using LSTM networks," in *Proc. IEEE Conf. Games*, 2023, pp. 1–4.

[13] Y. J. Choi, "Providing novel and useful data for game development using usability expert evaluation and testing," in *Proc. Imag. Visual. 6th Int. Conf. Comput. Graph.*, 2009, pp. 129–132, doi: 10.1109/CGIV.2009.51.

[14] H. Korhonen and E. M. I. Koivisto, "Playability heuristics for mobile games," in *Proc. 8th Conf. Hum.-Comput. Interact. With Mobile Devices Serv.*, 2006, pp. 9–16, doi: 10.1145/1152215.1152218.

[15] P. Moreno-Ger, J. Torrente, Y. G. Hsieh, and W. T. Lester, "Usability testing for serious games: Making informed design decisions with user data," *Adv. Hum.-Comput. Interact.*, vol. 2012, Nov. 2012, Art. no. e369637, doi: 10.1155/2012/369637.

[16] N. M. Diah, M. Ismail, S. Ahmad, and M. K. M. Dahari, "Usability testing for educational computer game using observation method," in *Proc. Int. Conf. Inf. Retrieval Knowl. Manage.*, 2010, pp. 157–161, doi: 10.1109/INFRKM.2010.5466926.

[17] I. Dobles, A. Martínez, and C. Quesada-López, "Comparing the effort and effectiveness of automated and manual tests," in *Proc. 14th Iberian Conf. Inf. Syst. Technol.*, 2019, pp. 1–6, doi: 10.23919/CISTI.2019.8760848.

[18] S. Iftikhar, M. Z. Iqbal, M. U. Khan, and W. Mahmood, "An automated model based testing approach for platform games," in *Proc. ACM/IEEE 18th Int. Conf. Model Driven Eng. Lang. Syst.*, 2015, pp. 426–435, doi: 10.1109/MODELS.2015.7338274.

[19] G. Lovreto, A. T. Endo, P. Nardi, and V. H. S. Durelli, "Automated tests for mobile games: An experience report," in *Proc. 17th Braz. Symp. Comput. Games Digit. Entertainment*, 2018, pp. 48–488, doi: 10.1109/SBGAMES.2018.00015.

[20] E. Guglielmi, S. Scalabrino, G. Bavota, and R. Oliveto, "Using gameplay videos for detecting issues in video games," *Empirical Softw. Eng.*, vol. 28, no. 6, Oct. 2023, Art. no. 136, doi: 10.1007/s10664-023-10365-0.

[21] I. S. W. B. Prasetya et al., "Navigation and exploration in 3D-game automated play testing," in *Proc. 11th ACM SIGSOFT Int. Workshop Automating TEST Case Des., Selection, Eval.*, 2020, pp. 3–9, doi: 10.1145/3412452.3423570.

[22] A. Albaghajati and M. Ahmed, "A co-evolutionary genetic algorithms approach to detect video game bugs," *J. Syst. Softw.*, vol. 188, Jun. 2022, Art. no. 111261, doi: 10.1016/j.jss.2022.111261.

[23] R. Ferdous, F. Kifetew, D. Prandi, I. S. W. B. Prasetya, S. Shirzadehhajimahmood, and A. Susi, "Search-based automated play testing of computer games: A model-based approach," in *Search-Based Software Engineering* (in Lecture Notes in Computer Science), U.-M. O'Reilly and X. Devroey, Eds., Berlin, Germany: Springer, 2021, pp. 56–71, doi: 10.1007/978-3-030-88106-1_5.

[24] Y. Zhao, E. Tang, H. Cai, X. Guo, X. Wang, and N. Meng, "A lightweight approach of human-like playtest for android apps," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering*, 2022, pp. 309–320, doi: 10.1109/SANER53432.2022.00047.

[25] K. Chang, B. Aytemiz, and A. M. Smith, "Reveal-more: Amplifying human effort in quality assurance testing using automated exploration," in *Proc. IEEE Conf. Games*, 2019, pp. 1–8, doi: 10.1109/CIG.2019.8848091.

[26] "Gym retro," 2016. Accessed: Jul. 9, 2023. [Online]. Available: https://openai.com/research/gym-retro

[27] A. Nantes, R. Brown, and F. Maire, "A framework for the semi-automatic testing of video games," in *Proc. AAAI Conf. Artif. Intell. Interactive Digit. Entertainment*, 2008, pp. 197–202, doi: 10.1609/aiide.v4i1.18697.

[28] M. Ferguson, S. Devlin, D. Kudenko, and J.A. Walker, "Imitating playstyle with dynamic time warping imitation," in *Proc. 17th Int. Conf. Found. Digit. Games*, 2022, pp. 1–11. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3555858.3555878
[29] C. Paduraru, M. Paduraru, and A. Stefanescu, "RiverGame - a game testing tool using artificial intelligence," in *Proc. IEEE Conf. Softw. Testing, Verification Validation*, 2022, pp. 422–432, doi: 10.1109/ICST53961.2022.00048.
[30] D. Lin, C.-P. Bezemer, and A. E. Hassan, "Identifying gameplay videos that exhibit bugs in computer games," *Empirical Softw. Eng.*, vol. 24, no. 6, pp. 4006–4033, Dec. 2019, doi: 10.1007/s10664-019-09733-6.
[31] O. Keehl and A. M. Smith, "Monster Carlo 2: Integrating learning and tree search for machine playtesting," in *Proc. IEEE Conf. Games*, 2019, pp. 1–8, doi: 10.1109/CIG.2019.8847989.
[32] O. Keehl and A. M. Smith, "Monster Carlo: An MCTS-based framework for machine playtesting unity games," in *Proc. IEEE Conf. Comput. Intell. Games*, 2018, pp. 1–8, doi: 10.1109/CIG.2018.8490363.
[33] J. Pfau, A. Liapis, G. N. Yannakakis, and R. Malaka, "Dungeons & replicants II: Automated game balancing across multiple difficulty dimensions via deep player behavior modeling," *IEEE Trans. Games*, vol. 15, no. 2, pp. 217–227, Jun. 2023, doi: 10.1109/TG.2022.3167728.
[34] M. R. Taesiri, M. Habibi, and M. A. Fazli, "A video game testing method utilizing deep learning," *CSI J. Comp. Sci. Eng.*, vol. 17, no. 2, pp. 26–33, 2022. [Online]. Available: https://jcse.ir/article/254
[35] M. R. Taesiri, F. Macklon, and C.-P. Bezemer, "CLIP meets GamePhysics: Towards bug identification in gameplay videos using zero-shot transfer learning," in *Proc. 19th Int. Conf. Mining Softw. Repositories*, 2022, pp. 270–281, doi: 10.1145/3524842.3528438.
[36] "Reddit - dive into anything," 2005. Accessed: Jul. 9, 2023. [Online]. Available: https://www.reddit.com/
[37] C. G. Ling, K. Tollmar, and L. Gisslén, "Using deep convolutional neural networks to detect rendered glitches in video games," in *Proc. 16th AAAI Conf. Artif. Intell. Interactive Digit. Entertainment*, 2020, pp. 66–73.
[38] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
[39] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," Apr. 2015, *arXiv:1409.1556*.
[40] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.
[41] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520, doi: 10.1109/CVPR.2018.00474.
[42] F. Xue, "Automated mobile apps testing from visual perspective," in *Proc. 29th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 577–581. 2022.[Online]. Available: https://dl.acm.org/doi/abs/10.1145/3395363.3402644
[43] R. Tufano, S. Scalabrino, L. Pascarella, E. Aghajani, R. Oliveto, and G. Bavota, "Using reinforcement learning for load testing of video games," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 2303–2314, doi: 10.1145/3510003.3510625.
[44] B. Wilkins, C. Watkins, and K. Stathis, "A metric learning approach to anomaly detection in video games," in *Proc. IEEE Conf. Games*, 2020, pp. 604–607, doi: 10.1109/CoG47356.2020.9231700.
[45] D. Chicco, "Siamese neural networks: An overview," in *Artificial Neural Networks* (Methods in Molecular Biology Series), H. Cartwright, Ed., Berlin, Germany: Springer, 2021, pp. 73–94, doi: 10.1007/978-1-0716-0826-5_3.
[46] B. Wilkins and K. Stathis, "World of bugs: A platform for automated bug detection in 3D video games," in *Proc. IEEE Conf. Games*, 2022, pp. 520–523, doi: 10.1109/CoG51982.2022.9893616.
[47] "FPS microgame," Unity Learn. Accessed: Dec. 06, 2019. [Online]. Available: https://learn.unity.com/project/fps-template
[48] M. Jaén-Vargas et al., "Effects of sliding window variation in the performance of acceleration-based human activity recognition using deep learning models," *PeerJ Comput. Sci.*, vol. 8, Aug. 2022, Art. no. e1052, doi: 10.7717/peerj-cs.1052.
[49] R. M. Schmidt, "Recurrent neural networks (RNNs): A gentle introduction and overview," Nov. 2019, *arXiv:1912.05911*.
[50] K. Team, "Keras documentation: TimeDistributed layer," Accessed: Jul. 7, 2023. [Online]. Available: https://keras.io/api/layers/recurrent_layers/time_distributed/
[51] Y. S. Chong and Y. H. Tay, "Abnormal event detection in videos using spatiotemporal autoencoder," in *Advances in Neural Networks - ISNN 2017* (in Lecture Notes in Computer Science), F. Cong, A. Leung, and Q. Wei, Eds., Berlin, Germany: Springer, 2017, pp. 189–196, doi: 10.1007/978-3-319-59081-3_23.
[52] Pattern Recognition and Machine Learning. 2006. Accessed: Jul. 09, 2023. [Online]. Available: https://link.springer.com/book/9780387310732
[53] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning* (Springer Series in Statistics). Berlin, Germany: Springer, 2009, doi: 10.1007/978-0-387-84858-7.
[54] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," Jan. 2017, *arXiv:1412.6980*.
[55] M. Hasan, J. Choi, J. Neumann, A. K. Roy-Chowdhury, and L. S. Davis, "Learning temporal regularity in video sequences," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 733–742, doi: 10.1109/CVPR.2016.86.
[56] "Sklearn.Metrics.Homogeneity_score," scikit-learn. Accessed: Jul. 9, 2023. [Online]. Available: https://scikit-learn/stable/modules/generated/sklearn.metrics.homogeneity_score.html
[57] OpenHogwarts, "OpenHogwarts/hogwarts," Jul. 2023, Accessed: Jul. 9, 2023. [Online]. Available: https://github.com/OpenHogwarts/hogwarts
[58] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, Feb. 2015, Art. no. 7540, doi: 10.1038/nature14236.

Elham Azizi received the master's degree in computer science from Ontario Tech University, Oshawa, ON, Canada, in 2023.

Her research interests include applications of deep learning in game analytics and the multidisciplinary uses of machine learning in various fields, including medicine and animal welfare.

Loutfouz Zaman received the Ph.D. degree in computer science from York University, Toronto, Canada, in 2016. He leads research in game analytics and human–computer interaction in games with Ontario Tech University, Oshawa, ON, Canada. He supervises graduate students on various projects, including deep learning for bug detection, visual game analytics, and games user research.